HW7 CRUD + Cal BMI

```
CREATE TABLE emp2 (
id int(11) NOT NULL AUTO_INCREMENT PRIMARY KEY,
name varchar(255) NOT NULL,
email varchar(255) NOT NULL,
phone varchar(16) DEFAULT NULL,
address text DEFAULT NULL,
weight REAL NOT NULL,
height REAL NOT NULL
)
```

```html
        </form>
        <script>
                async function ins_data() {
                        const rawResponse = await fetch("/create", {
                                method: "post",
                                headers: {
                                        Accept: "application/json",
                                        "Content-Type": "application/json",
                                },
                                body: JSON.stringify({
                                        name: document.getElementById("iName").value,
                                        email: document.getElementById("iEmail").value,
                                        phone: document.getElementById("iPhone").value,
                                        address: document.getElementById("iAddress").value,
                                        weight: document.getElementById("iWeight").value,
                                        height: document.getElementById("iHeight").value,
                                }),
                        });
                        const content = await rawResponse.json();
                        console.log(content);
                }
        </script>
</doctype>
```

templates > add.html > doctype

```html
<doctype html>
        <title>Add User - Python Flask MySQL CRUD</title>
        <h2>Add User</h2>
        <form>
                <dl>
                        <input name="inputName" id="iName" value="" type="text" placeholder="Name" autocomplete="off"
                                required />
                        <p>
                                <input name="inputEmail" id="iEmail" value="" type="text" placeholder="Email"
                                        autocomplete="off" required />
                        </p>
                        <p>
                                <input name="inputPhone" id="iPhone" value="" type="text" placeholder="Phone"
                                        autocomplete="off" required />
                        </p>
                        <p>
                                <input name="inputaddress" id="iAddress" value="" type="text" placeholder="Address"
                                        autocomplete="off" required />
                        </p>
                        <p>
                                <input name="inputweight" id="iWeight" value="" type="text" placeholder="Weight"
                                        autocomplete="off" required />
                        </p>
                        <p>
                                <input name="inputheight" id="iHeight" value="" type="text" placeholder="Height"
                                        autocomplete="off" required />
                        </p>
                </dl>
                <p>
                        <button onclick="ins_data()">save</button>
                </p>
```

model.dart > Model > Model_

```dart
class Model {
  final String id;
  final String name;
  final String address;
  final String email;
  final double weight;
  final double height;

  Model._({
    required this.id,
    required this.name,
    required this.address,
    required this.email,
    required this.weight,
    required this.height
});
  factory Model.fromJson(Map<String, dynamic> json) {
    return Model._(
      id: json['id'],
      name: json['name'],
      address: json['address'],
      email: json['email'],
      weight: json['weight'],
      height: json['height'],
    );
  }
}
```

mymain.dart > ...

```dart
import 'package:flutter/material.dart';
import 'dart:async';
import 'dart:convert';
import 'package:http/http.dart' as http;

Run | Debug | Profile
main() {
  runApp(const ShowInf());
}

class ShowInf extends StatefulWidget {
  const ShowInf({super.key});

  @override
  State<ShowInf> createState() => _ShowInfState();
}

class _ShowInfState extends State<ShowInf> {
  List list = [];
  final TextEditingController _nameController = TextEditingController();
  final TextEditingController _addressController = TextEditingController();
  final TextEditingController _emailController = TextEditingController();
  final TextEditingController _phoneController = TextEditingController();
  final TextEditingController _weightController = TextEditingController();
  final TextEditingController _heightController = TextEditingController();

  Future<String> listData() async {
    var response = await http.get(Uri.http('10.0.2.2:80', 'emp'),
        headers: {"Accept": "application/json"});
    print("Response status: ${response.statusCode}");    Don't in
    print("Response body: ${response.body}");    Don't invoke 'pr
    setState(() {
      list = jsonDecode(response.body);
    });
    return 'Success';
  }

  @override
  void initState() {
    super.initState();
    listData();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('DB Test'),    Use 'const' with the construct
      ), // AppBar
      body: Center(
        child: ListView.builder(
          itemCount: list.length,
          itemBuilder: (BuildContext context, int index) {
            return Card(
              child: ListTile(
    leading: Icon(Icons.person),    Use 'const' with the constructor
    title: Text("Name: ${list[index]['name']}"),
    subtitle: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text("Email: ${list[index]['email']}"),
        Text("Phone: ${list[index]['phone']}"),
        Text("Address: ${list[index]['address']}"),
        Text("Weight: ${list[index]['weight'].toStringAsFixed(2)} kg"),
        Text("Height: ${list[index]['height'].toStringAsFixed(2)} cm"),
        Text("BMI: ${calBmi(list[index]['weight'], list[index]['height'])}"),
        Text("BMI Type: ${calBmitype(calBmi(list[index]['weight'], list[index]['height']))}"),
        Text("Weight Status: ${toBenormal(list[index]['weight'], list[index]['height'])}"),
      ],
    ), // Column
              // leading: Text(list[index]['id'].toString()),
              trailing: Wrap(
                // spacing: 0,
                children: [
                  Image.asset(
        bmitypetoImage(calBmitype(calBmi(list[index]['weight'], list[index]['height']))),
        height: 50,  // Adjust size as needed
      ), // Image.asset
                  IconButton(
                    onPressed: () {
                      Map data = {
                        'id': list[index]['id'],
                        'name': list[index]['name'],
                        'address': list[index]['address'],
                        'email': list[index]['email'],
                        'phone': list[index]['phone'],
                        'weight': list[index]['weight'],
                        'height': list[index]['height']
                      };
                      _showedit(data);
                    },
                    icon: Icon(Icons.edit),    Use 'const' with the constructor to improve
                    color: Colors.green,
                  ), // IconButton
                  IconButton(
                      onPressed: () => _showDel(list[index]["id"]),
                      icon: const Icon(
                        Icons.delete_outline,
                        color: Colors.red,
                      )), // Icon // IconButton
                ],
              ), // Wrap
            ), // ListTile
          ); // Card
        },
      ), // ListView.builder
    ), // Center
    floatingActionButton: FloatingActionButton(
      onPressed: () {
        _addNewDialog();
      },
      child: const Icon(Icons.add),
    ), // FloatingActionButton
  ); // Scaffold
}

Future<void> _addNewDialog() async {
  return showDialog(
      barrierDismissible: false,
      context: context,
      builder: (BuildContext context) {
        return AlertDialog(
          title: const Text('Add data'),
          content: SingleChildScrollView(
              child: ListBody(
            children: <Widget>[
              TextField(
                controller: _nameController,
                decoration: const InputDecoration(
                    labelText: 'Name', hintText: "Enter emp Name"), /
              ), // TextField
              TextField(
                controller: _emailController,
                decoration: const InputDecoration(
                    labelText: 'Email', hintText: "Enter emp Email"),
              ), // TextField
              TextField(
                controller: _phoneController,
                decoration: const InputDecoration(
                    labelText: 'Phone'. hintText: "Enter emp Phone").
              ), // TextField
              TextField(
                controller: _addressController,
                decoration: const InputDecoration(
                    labelText: 'Address', hintText: "Enter emp Address"),
              ), // TextField
              TextField(
                controller: _weightController,
                keyboardType: TextInputType.number,
                decoration: const InputDecoration(
                    labelText: 'Weight', hintText: "Enter emp Weight"), //
              ), // TextField
              TextField(
                controller: _heightController,
                keyboardType: TextInputType.number,
                decoration: const InputDecoration(
```

```yaml
flutter:
  assets:
    - assets/images/
```

```
156                     labelText: 'Height', hintText: "Enter emp Height"),
157                  ), // TextField
158                  const Text('กรอกข้อมูลให้เรียบร้อยแล้วกด ยืนยัน'),
159                ], // <Widget>[]
160              )), // ListBody // SingleChildScrollView
161              actions: <Widget>[
162                TextButton(
163                    onPressed: () {
164                      add_data();
165                      Navigator.of(context).pop();
166                    },
167                    child: Text('ยืนยัน')), // TextButton    Use 'const' with
168              ], // <Widget>[]
169            ); // AlertDialog
170          });
171  }
172
173  Future<void> _showDel(int id) async {
174    return showDialog<void>(
175        barrierDismissible: false,
176        context: context,
177        builder: (BuildContext context) {
178          return AlertDialog(
179            title: Text('ลบข้อมูล ${id}'),    Unne
180            content: SingleChildScrollView(    U
181                child: ListBody(    Use 'const'
182              children: <Widget>[    Use 'const'
183                Text('ยืนยันการลบข้อมูล กด ยืนยัน'),
184              ], // <Widget>[]
185            )), // ListBody // SingleChildScroll
186            actions: <Widget>[
187              TextButton(
188                  onPressed: () {
189                    del_data(id);
190                    Navigator.of(context).pop();
191                  },
192                  child: Text('ยืนยัน')), // TextB
193            ], // <Widget>[]
194          ); // AlertDialog
195        });
196  }
197
198  void add_data() async {    The variable name 'add_data' isn't a lowe
199    Map data = {
200      'name': _nameController.text,
201      'email': _emailController.text,
202      'phone': _phoneController.text,
203      'address': _addressController.text,
204      'weight': _weightController.text,
205      'height': _heightController.text
206    };
207    var body = jsonEncode(data);
208    var response = await http.post(Uri.http('10.0.2.2:80', 'create'),
209        headers: {
210          "Content-Type": "application/json",
211          "Accept": "application/json"
212        },
213        body: body);
214    print("Response status: ${response.statusCode}");    Don't invoke
215    print("Response body: ${response.body}");    Don't invoke 'print'
216    listData();
217  }
218
219  void del_data(int id) async {    The variable name 'del_data' isn't a low
220    var response = await http.delete(Uri.http('10.0.2.2:80', 'delete/$id'),
221        headers: {
222          "Content-Type": "application/json; charset=UTF-8",
223          "Accept": "application/json"
224        });
225    print("Response status: ${response.statusCode}");    Don
226    print("Response body: ${response.body}");    Don't invok
227    listData();
228  }
229
230  Future<void> _showedit(Map data) async {
231    _nameController.text = data['name'];
232    _emailController.text = data['email'];
233    _phoneController.text = data['phone'];
234    _addressController.text = data['address'];
235    _weightController.text = data['weight'].toString();
236    _heightController.text = data['height'].toString();
237    return showDialog<void>(
238        barrierDismissible: false,
239        context: context,
240        builder: (BuildContext context) {
241          return AlertDialog(
242            title: const Text('ทดสอบการ Edit'),
```

```
243              content: SingleChildScrollView(
244                  child: ListBody(
245                children: <Widget>[
246                  TextField(
247                    controller: _nameController,
248                    decoration: const InputDecoration(
249                        labelText: 'Name', hintText: "Enter emp Name"), // Inp
250                  ), // TextField
251                  TextField(
252                    controller: _emailController,
253                    decoration: const InputDecoration(
254                        labelText: 'Email', hintText: "Enter emp Email"), // I
255                  ), // TextField
256                  TextField(
257                    controller: _phoneController,
258                    decoration: const InputDecoration(
259                        labelText: 'Phone', hintText: "Enter emp Phone"), // I
260                  ), // TextField
261                  TextField(
262                    controller: _addressController,
263                    decoration: const InputDecoration(
264                        labelText: 'Address', hintText: "Enter emp Address"),
265                  ), // TextField
266                  TextField(
267                    controller: _weightController,
268                    decoration: const InputDecoration(
269                        labelText: 'Weight', hintText: "Enter emp Weight"),
270                  ), // TextField
271                  TextField(
272                    controller: _heightController,
273                    decoration: const InputDecoration(
274                        labelText: 'Height', hintText: "Enter emp Height"),
275                  ), // TextField
276                  const Text('ปรับปรุงข้อมูลให้เรียบร้อยแล้วกด ยืนยัน'),
277                ], // <Widget>[]
278              )), // ListBody // SingleChildScrollView
279              actions: <Widget>[
280                TextButton(
281                    onPressed: () {
282                      edit_data(data['id']);
283                      Navigator.of(context).pop();
284                    },
285                    child: Text('ยืนยัน')), // TextButton    Use 'const' with
286              ], // <Widget>[]
287            ); // AlertDialog
288          });
289  }
290
291  void edit_data(id) async {    The variable name 'edit_data' isn't a lowe
292    Map data = {
293      'id': id,
294      'name': _nameController.text,
295      'email': _emailController.text,
296      'phone': _phoneController.text,
297      'address': _addressController.text,
298      'weight': double.tryParse(_weightController.text),
299      'height': double.tryParse(_heightController.text)
300    };
301
302    // Debug print to see the data being sent
303    print("Sending data to server: $data");    Don't invoke 'print' in pro
304
305    var body = jsonEncode(data);
306    print("Encoded JSON: $body");    Don't invoke 'print' in production co
307
308    try {
309      var response = await http.put(Uri.http('10.0.2.2:80', 'update/$id'),
310          headers: {
311            "Content-Type": "application/json; charset=UTF-8",
312          },
313          body: body);
314
315      print("Response status: ${response.statusCode}");    Don't invoke 'pr
316      print("Response body: ${response.body}");    Don't invoke 'print' in
317
318      if (response.statusCode == 500) {
319        print("Server error details: ${response.body}");    Don't invoke 'p
320      }
321
322      listData();
323    } catch (e) {
324      print("Error during update: $e");    Don't invoke 'print' in producti
325    }
326  }
327  String calBmi(weight,height) {
328    double weightfloat = double.tryParse(weight.toString()) ?? 0.0;
329    double heightfloat = double.tryParse(height.toString()) ?? 0.0;
330    double bmi = weightfloat / ((heightfloat / 100) * (heightfloat / 100));
331    return bmi.toStringAsFixed(2);
332  }
333  String calBmitype(bmi) {
334    double bmifloat = double.tryParse(bmi.toString()) ?? 0.0;
```

```
    if (bmifloat < 18.5) {
      return 'Underweight';
    } else if (bmifloat < 22.9) {
      return 'Normal';
    } else if (bmifloat < 24.9) {
      return 'Risk to Overweight';
    } else if (bmifloat < 29.9) {
      return 'Overweight';
    } else {
      return 'Obesity';
    }
  }
  String bmitypetoImage(bmi_type){    The variat
    if (bmi_type == "Underweight"){
      return "assets/images/bmi-1.png";
    }
    else if (bmi_type == "Normal"){
      return "assets/images/bmi-2.png";
    }
    else if (bmi_type == "Risk to Overweight"){
      return "assets/images/bmi-3.png";
    }
    else if (bmi_type == "Overweight"){
      return "assets/images/bmi-4.png";
    }
    else{
      return "assets/images/bmi-5.png";
    }
}
String toBenormal(weight,height){
  late double weighttonormal;
  late double diff;
  double nowbmi = weight / ((height/100)*(height/100));
  if (nowbmi < 18.5){
    weighttonormal = 18.5*height*height/10000;
    diff = weighttonormal - weight;
    diff = double.parse((diff).toStringAsFixed(2));
    return "You need to gain atleast ${diff} kg to be Normal";
  }
  else if (nowbmi >= 22.9){
    weighttonormal = 22.9*height*height/10000;
    diff = weight - weighttonormal;
    diff = double.parse((diff).toStringAsFixed(2));
    return "You need to lose atleast ${diff} kg to be Normal";
  }
  else{
    return "Good Health";
  }
}
}
```

server.py > ...

```
import pymysql
from werkzeug.wrappers import Request, Response
from flask import flash, render_template, request
from flask import jsonify
from flask import Flask
from flaskext.mysql import MySQL

app = Flask(__name__)
mysql = MySQL()
app.config['MYSQL_DATABASE_USER'] = 'root'
app.config['MYSQL_DATABASE_PASSWORD'] = ''
app.config['MYSQL_DATABASE_DB'] = 'emp'
app.config['MYSQL_DATABASE_HOST'] = 'localhost'
mysql.init_app(app)
Ctrl+I to chat, Ctrl+K to generate
@app.route("/")
def hello():
    return """
    Flask API !<br>
    <a href='/new_user'>add</a>insert new<br>
    <a href='/emp'>emp</a>Show all<br>
    <b>/create</b> Insert new Record<br>
    <b>/emp/<id></b> get by ID<br>
    <b>/update/<id></b> Edit info<br>
    <b>/delete/<id></b> Delete by  ID<br>
```

```
    """
@app.route('/new_user')
def add_user_view():
    return render_template('add.html')

@app.route('/create', methods=['POST'])
def create_emp():
    try:
        _json = request.json
        print(_json)
        _name = _json['name']
        _email = _json['email']
        _phone = _json['phone']
        _address = _json['address']
        _weight = _json['weight']
        _height = _json['height']
        if _name and _email and _phone and _address and _weight and _height and request.method == 'POST':
            conn = mysql.connect()
            cursor = conn.cursor(pymysql.cursors.DictCursor)
            sqlQuery = "INSERT INTO emp2(name, email, phone, address, weight, height) VALUES(%s, %s, %s, %s, %s, %s)"
            bindData = (_name, _email, _phone, _address, _weight, _height)
            cursor.execute(sqlQuery, bindData)
            conn.commit()
            respone = jsonify('Employee added successfully!')
            respone.status_code = 200
            #flash('User added successfully!')
            #redirect('/')
            return respone
        else:
            return showMessage()    "showMessage" is not defined
    except Exception as e:
        print(e)
    finally:
        if cursor:
            cursor.close()
        if conn:
            conn.close()

@app.route('/emp')
def emp():
    try:
        conn = mysql.connect()
        cursor = conn.cursor(pymysql.cursors.DictCursor)
        cursor.execute("SELECT id, name, email, phone, address, weight, height FROM emp2")
        empRows = cursor.fetchall()
        respone = jsonify(empRows)
        respone.status_code = 200
        return respone
    except Exception as e:
        print(e)
    finally:
        if cursor:
            cursor.close()
        if conn:
            conn.close()

@app.route('/emp/<int:emp_id>')
def emp_details(emp_id):
    try:
        conn = mysql.connect()
        cursor = conn.cursor(pymysql.cursors.DictCursor)
        cursor.execute("SELECT id, name, email, phone, address, weight, height FROM emp2 WHERE id =%s", emp_id)
        empRow = cursor.fetchone()
        respone = jsonify(empRow)
        respone.status_code = 200
        return respone
    except Exception as e:
        print(e)
    finally:
        if cursor:
            cursor.close()
        if conn:
            conn.close()

@app.route('/update/<int:emp_id>', methods=['PUT'])
def update_emp(emp_id):
    try:
        _json = request.json
        #_id = _json['id']
        _name = _json['name']
        _email = _json['email']
        _phone = _json['phone']
        _address = _json['address']
        _weight = _json['weight']
        _height = _json['height']
        if _name and _email and _phone and _address and _weight and _height and emp_id and request.method == 'PUT':
            sqlQuery = "UPDATE emp2 SET name=%s, email=%s, phone=%s, address=%s, weight=%s, height=%s WHERE id=%s"
            bindData = (_name, _email, _phone, _address, _weight, _height, emp_id,)
            conn = mysql.connect()
            cursor = conn.cursor()
            cursor.execute(sqlQuery, bindData)
            conn.commit()
            respone = jsonify('Employee updated successfully!')
            respone.status_code = 200
            return respone
        else:
            return showMessage()    "showMessage" is not defined
    except Exception as e:
        print(e)
    finally:
        if cursor:
            cursor.close()
        if conn:
            conn.close()

@app.route('/delete/<int:emp_id>', methods=['DELETE'])
def delete_emp(emp_id):
    try:
        conn = mysql.connect()
        cursor = conn.cursor()
        cursor.execute("DELETE FROM emp2 WHERE id =%s", (emp_id,))
        conn.commit()
        respone = jsonify('Employee deleted successfully!')
        respone.status_code = 200
        return respone
    except Exception as e:
        print(e)
    finally:
        if cursor:
            cursor.close()
        if conn:
            conn.close()

if __name__ == '__main__':
    from werkzeug.serving import run_simple
    app.debug = True
    run_simple('localhost', 80, app) # chang
```

globals.dart

```
// globals.dart
library my_project.globals;

String username = "";
String card_number = "0";
```

```
globals.username = username;
globals.card_number = user['card_number'];
```